Fundamentals of Deep Learning and Programming

Lecture 1: Introduction to Python Programming

Fall 2025

Adjunct Professor: Donghoon Kang

Python

Version: 3.10

Advantages

- Easy to learn,
- suitable for developing deep learning SW, web applications

Disadvantages

- Slow execution time
- High memory usage -> unsuitable for developing mobile applications

Setting up Your Deep Learning Environments

For Windows 

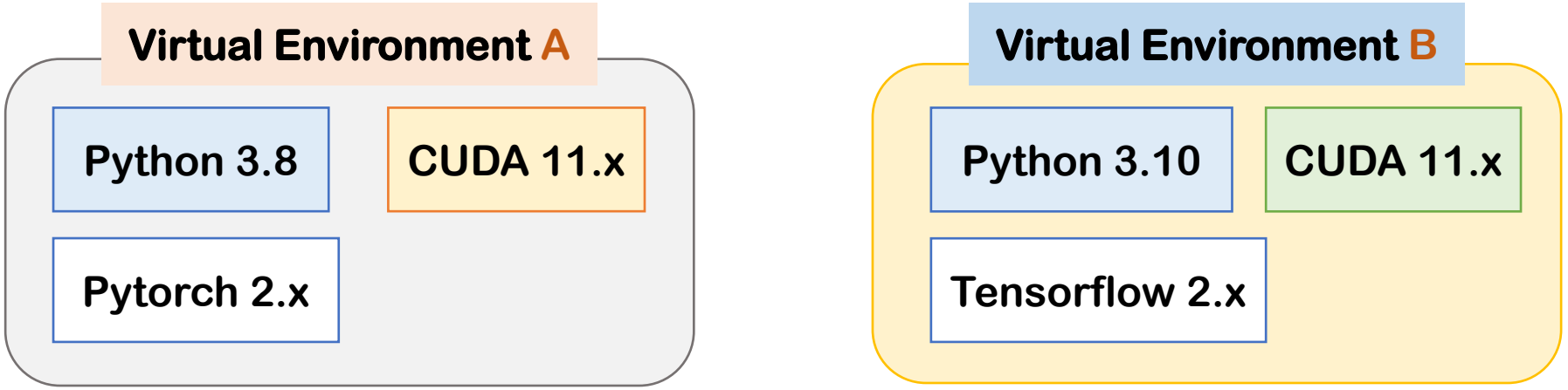
1. Install Anaconda (includes Python)
2. Install Visual Studio Code (=VS Code)
3. Create Virtual Environments in Anaconda
    1) Install Pytorch (that includes CUDA)
    2) Connect Virtual Environments in Anaconda to VS Code

Setting up Your Deep Learning Environments

For ubuntu

1. Install Anaconda (includes Python)
2. Install CUDA Toolkit
3. Install cuDNN
4. Install Visual Studio Code (=VS Code)
5. Create Virtual Environments in Anaconda
    1) Install Pytorch (that includes CUDA)
    2) Connect Virtual Environments in Anaconda to VS Code

Virtual Environments using Anaconda



- Isolation
- Project-specific configuration

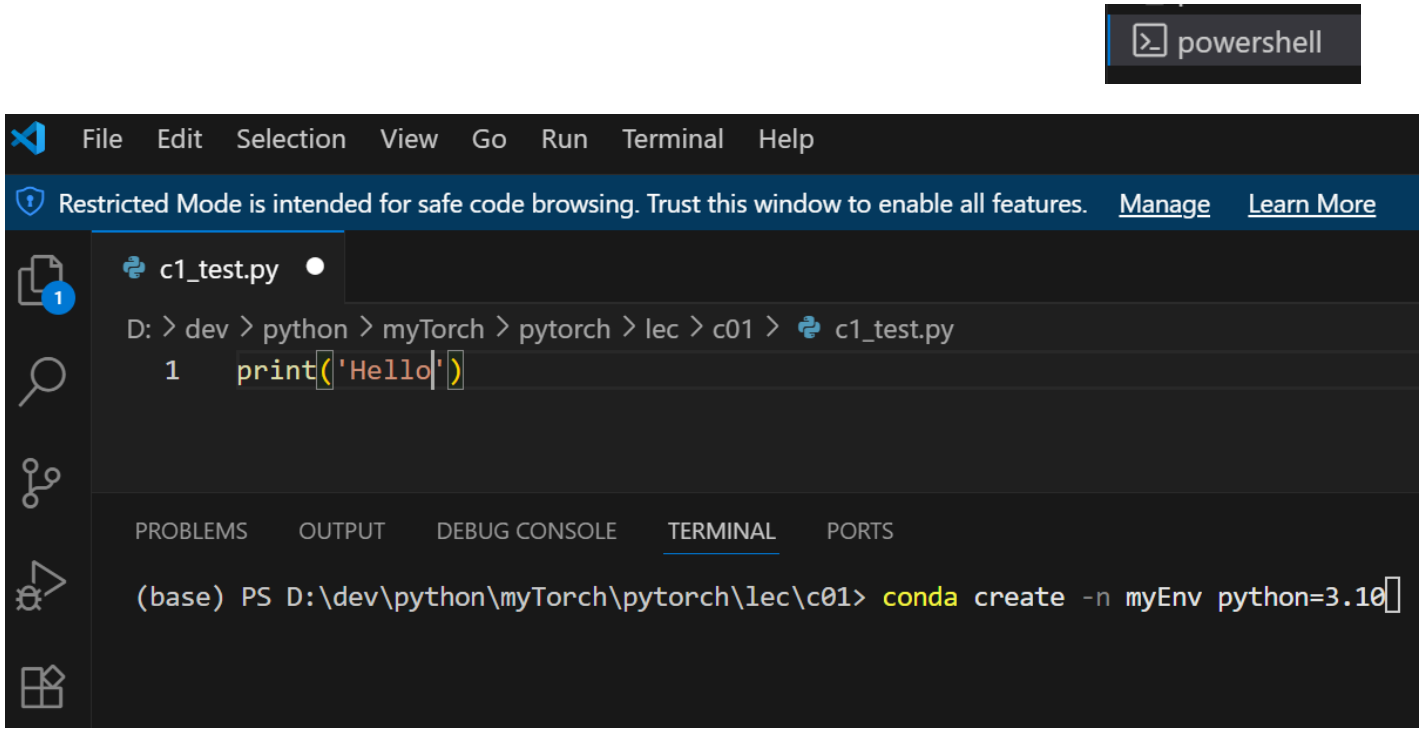
Virtual Environments using Anaconda

1. Create Virtual Environment

```
conda create -n myEnv python=3.10
```

2. Delete the Virtual Environment

```
conda remove --name myEnv --all
```



Virtual Environments using Anaconda

3. Activate Virtual Environment

```
conda activate myEnv
```

```
(base) PS D:\dev\python\myTorch\pytorch\lec\c01> conda activate myEnv
(myEnv) PS D:\dev\python\myTorch\pytorch\lec\c01>
```

4. Deactivate Virtual Environment

```
conda deactivate
```

```
(myEnv) PS D:\dev\python\myTorch\pytorch\lec\c01> conda deactivate
(base) PS D:\dev\python\myTorch\pytorch\lec\c01>
```

5. List All Virtual Environments

```
conda env list
```

```
(base) PS D:\dev\python\myTorch\pytorch\lec\c01> conda env list

# conda environments:
#
base                  * C:\Users\mo\anaconda3
myDataAnalysis          C:\Users\mo\anaconda3\envs\myDataAnalysis
myEnv                   C:\Users\mo\anaconda3\envs\myEnv
myEnvPytorch            C:\Users\mo\anaconda3\envs\myEnvPytorch
myEnvTensorflow         C:\Users\mo\anaconda3\envs\myEnvTensorflow
myPytorch               C:\Users\mo\anaconda3\envs\myPytorch
```

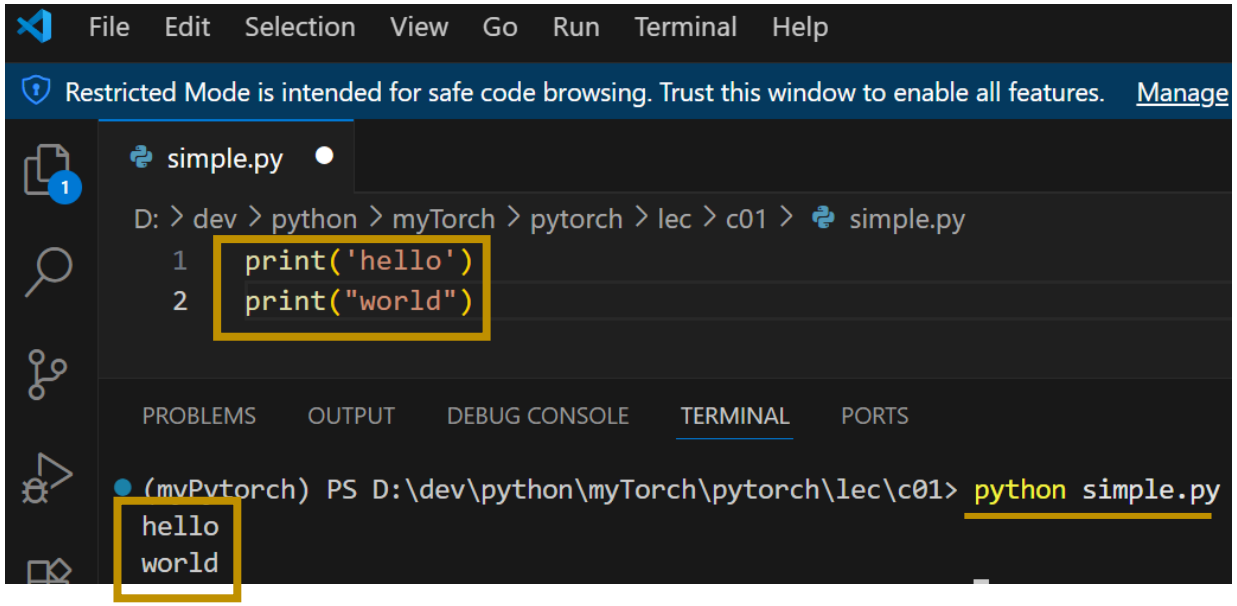
Python

Beginner level skills

print( ) function

```
print('hello')
print("world")
```

```
hello
world
```




```
print('hello', end = ' ')
print("world")
```

```
hello   world
```

```
print('hello', end = ' test ')
print("world")
```

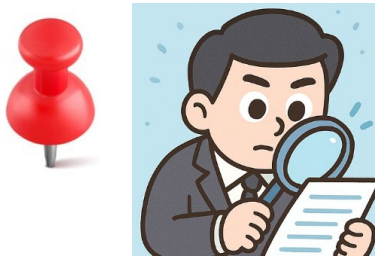
```
hello test world
```

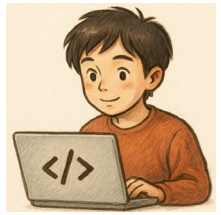
```
print('hello', end = '\n')
print("world")
```

```
hello
world
```

Built-in Object Types in Python

- numbers : -5.23 # float
- string : 'egg'
- bool : a == b # True
- list : [1, 3, -4]
- tuple : (1, 3, -4)
- set : {1, 'hello'}
- dictionary: {'age': 25}





Numbers

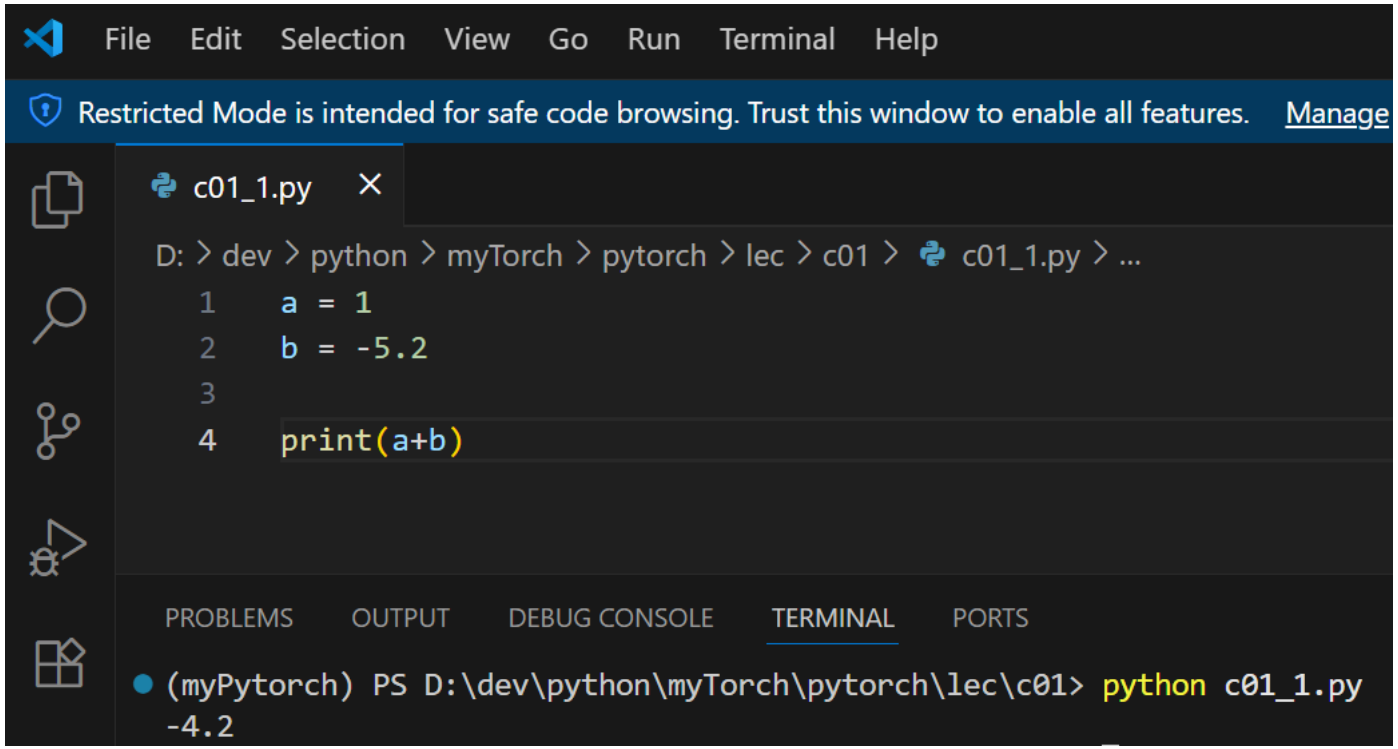
```
a = 1       # int
b = -5.2    # float
print(type(a))
print(type(b))
```

```
<class 'int'>
<class 'float'>
```

Numbers

```python
a = 1       # int
b = -5.2    # float

print(a)
print(b)
print(a+b) # float
```

```
1
-5.2
-4.2
```



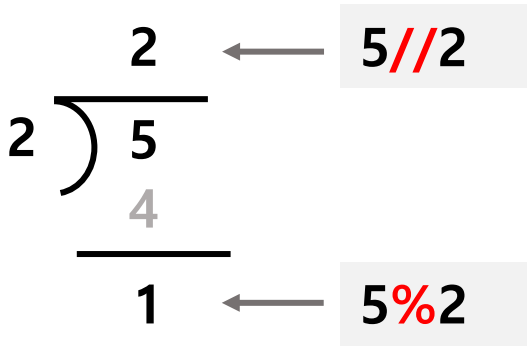

```python
a = 1
    b = -5.2    # error
```

```
    b = -5.2
IndentationError: unexpected indent
```

Numbers

```
print(5*2)   #10
print(5/2)   #2.5
print(5//2) #2
print(5%2)   #1

Print(5**2) #5^2
```

```
10
2.5
2
1
25
```



Numbers

```
a = 5

a += 2    # a = a + 2
a -= 2    # a = a - 2
a *= 2    # a = a * 2
a /= 2    # a = a / 2
a //= 2    # a = a // 2
a %= 2    # a = a % 2
a **= 2   # a = a ** 2
```

`a = 1` # we **assign** a variable **a** to an **object** (its value is 1)

variable **a**

memory address **1984297107696**

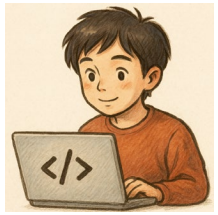
object **1**

```
a = 1
print(id(a))
```

```
1984297107696
```

memory address

In Python, **everything that exists in memory** is an "**object**".



String Type

```
a = 'cat'
b = "apple"
```

String

```
a = 'Hello '
b = "world "

print(a)
```

```
Hello
```

```
print(a + 3)  # error
```

```
    print(a+3)
TypeError: can only concatenate str (not "int") to str
```

```
a = 'cat'
print(len(a))
```

```
3
```

```
print(a + b)
print(b * 3)
```

```
Hello world
world world world
```

String

indexing

```
b = "world "
print(b[0])   #w
print(b[-1])  #d
print(b[-3])  #r
```

```
w
d
r
```

index of b

w	o	r	l	d
0	1	2	3	4
-5	-4	-3	-2	-1

Slicing (=extracting sequences)

```
b[:4] # b[0], b[1], b[2], b[3]: slice of a from starting index 0 to 3
a[2:] # a[2], a[3], ... slice of a starting index 2 to the end
a[:]    # a == a[:]
```

start

end

Slicing (=extracting sequences)

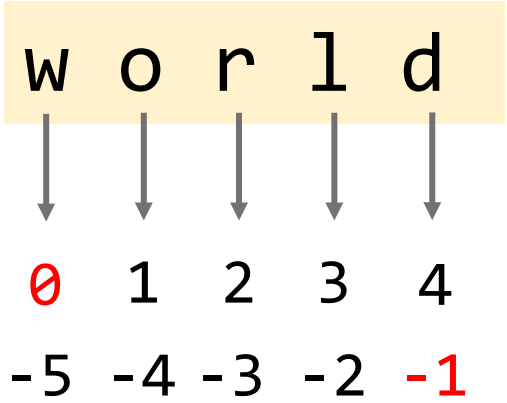
```
a = 'world'
print(a[1:3]) # or
print(a[:3])  # wor
print(a[2:])  # rld
print(a[:])   # world
print(a[-2:]) # ld
```

a[1], a[2] ~~a[3]~~

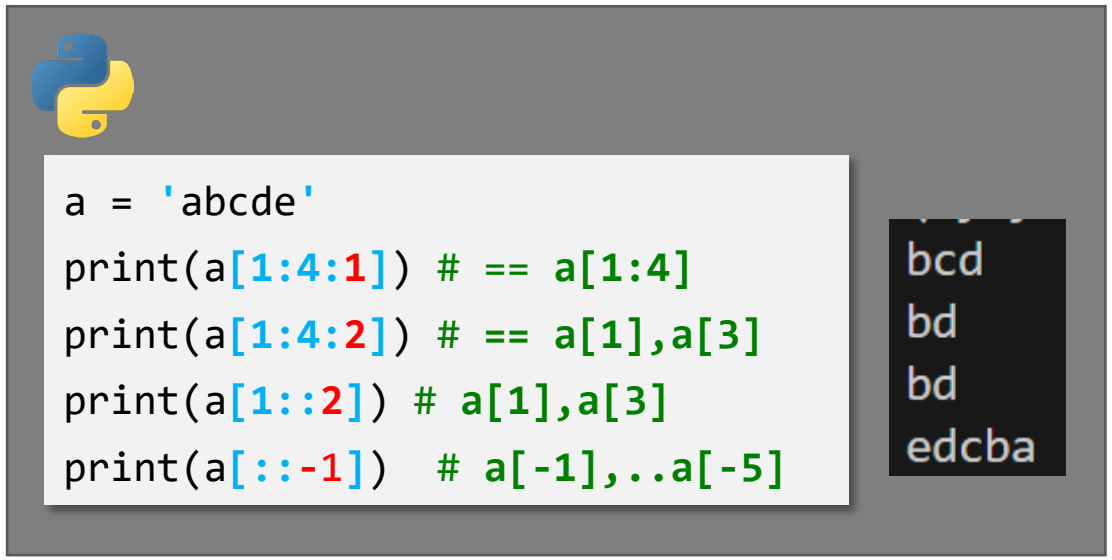
`a[1:3]` #slice of **a** **from index 1 to 2**

starting at index 1

ending before index 3 (**not 3**)

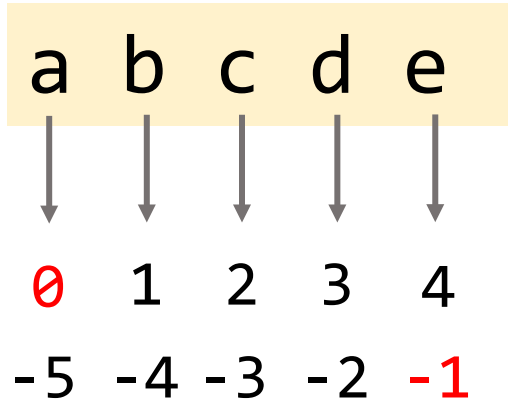


Slicing (=extracting sequences)



```
a = 'abcde'
print(a[1:4:1]) # == a[1:4]
print(a[1:4:2]) # == a[1],a[3]
print(a[1::2]) # a[1],a[3]
print(a[::-1])  # a[-1],..a[-5]
```

```
bcd
bd
bd
edcba
```




```
a[start:end+1:step]
```

String

```
a = 'Hello '
b = "world"

print(a)
print(a + b)
```

```
Hello
Hello world
```

	w	o	r	l	d
index of b	0	1	2	3	4
	-5	-4	-3	-2	-1

```
a[0] = 'd'  # cause error
```

```
    a[0] = 'd'
    ~^^^
TypeError: 'str' object does not support item assignment
```

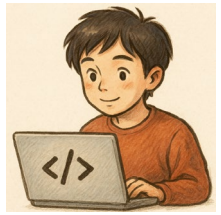
Str is immutable, meaning you can't change a string's content.

```
a = 'd' + a[1:]   # 'dello'
print (a)
```

```
dello
```

But this is OK.
a = 'd' + a[1:] does not modify the original string.

Instead, it creates a new string and reassign the variable a to refer to this new string



bool Type

```
a = True
b = False

c = 8 > 3 # True
d = 8 < 3 # False

print(c)
print(d)
```

```
True
False
```

bool

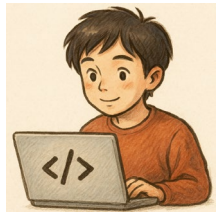
```
print(8 > 3)
print(8 < 3)
print(8 == 3)
print(8 != 3)
print(5 >= 5)
print(8 <= 3)
```

```
True
False
False
True
True
False
```

```
a = True or False
b = True and False
c = not (8<5) # not False
```

```
print(a)
print(b)
print(c)
```

```
True
False
True
```



List

```
[1, 3, 5]
['cat', 3.4]
['dog', [-2, 7]]
```

Tuple

```
(1, 3, 5)
('cat', 3.4)
('dog', [-2, 7])
```

List

```
a = [1, 2, 3]
b = [4, 5, 6]
```

```
print(a + b) # concatenate a and b
```

```
[1, 2, 3, 4, 5, 6]
```

```
print(a*2)
```

```
[1, 2, 3, 1, 2, 3]
```

```
len(a)   # 3
```

Tuple

```
e = (1, 2, 3)
f = (4, 5, 6)
```

```
print(e + f) # concatenate e and f
```

```
(1, 2, 3, 4, 5, 6)
```

```
print(e*2)
```

```
(1, 2, 3, 1, 2, 3)
```

```
len(e)   # 3
```

List

```
a = [1, 3, 5]
b = ['cat', 3.4]
c = ['dog', [-2, 7]]
d = []
```

```
print(a[0],a[1],a[-1])
print(b[0])
```

```
1 3 5
cat
```

```
len(a)   # 3
```

```
print(b + c)
```

```
['cat', 3.4, 'dog', [-2, 7]]
```

Tuple

```
e = (1, 3, 5)
f = ('cat', 3.4)
g = ('dog', [-2, 7])
h = ()
```

```
print(e[0],e[1],e[-1])
print(f[0])
```

```
1 3 5
cat
```

```
len(e)   # 3
```

```
print(f + g)
```

```
('cat', 3.4, 'dog', [-2, 7])
```

List

```
a = list(1,3,5)     # error
```

```
    a = list(1,3,5)
TypeError: list expected at most 1 argument, got 3
```

```
a = list([1,3,5])
b = list((1,3,5))
c = list('cat')
d = list(range(1,4))
```

```
[1, 3, 5]
[1, 3, 5]
['c', 'a', 't']
[1, 2, 3]
```

```
print(a)
print(b)
print(c)
print(d)
```

Tuple

```
a = tuple(1,3,5)     # error
```

```
    e = tuple(1, 3, 5)
TypeError: tuple expected at most 1 argument, got 3
```

```
e = tuple([1,3,5])
f = tuple((1,3,5))
g = tuple('cat')
h = tuple(range(1,4))
```

```
(1, 3, 5)
(1, 3, 5)
('c', 'a', 't')
(1, 2, 3)
```

```
print(e)
print(f)
print(g)
print(h)
```

Mutable vs. Immutable

IMPORTANT

List

```
a = [1, 3, 5]
a[0] = 9  # OK. List is a mutable (=changeable) type
print(a)
```

```
9 3 5
```

Tuple

```
e = (1, 3, 5)
e[0] = 9 #  error
         # Tuple is an immutable (=unchangeable) type
```

```
    e[0] = 9
TypeError: 'tuple' object does not support item assignment
```

Slicing

List

```
a = [1, 3, 5, 7]
print(a[1:2])
print(a[:2])
print(a[1:])
print(a[:])
```

```
[3]
[1,3]
[3,5,7]
[1,3,5,7]
```

Tuple

```
b = (1, 3, 5, 7)
print(b[1:2])
print(b[:2])
print(b[1:])
print(b[:])
```

```
(3)
(1,3)
(3,5,7)
(1,3,5,7)
```

```
a = [1, 3, 5, 7]
```

Index of a	0	1	2	3
	-4	-3	-2	-1

Nesting

List

```
a = [[1, 2, 3],
     [-2, -3, 10]]

print(a[1])
print(a[1][2])
```

```
[-2, -3, 10]
10
```

```
a[1][2] = -5
print(a[1][2])
```

```
-5
```

Tuple

```
b = ((1, 2, 3),
     (-2, -3, 10))

print(b[1])
print(b[1][2])
```

```
(-2, -3, 10)
10
```

```
b[1][2] = -5  # error
print(b[1][2])
```

tuple is an **immutable** type

```
    b[1][2] = -5
TypeError: 'tuple' object does not support item assignment
```

Tuple

```
a = (1, 3, 5)  # tuple
b =  1, 3, 5   # tuple
```

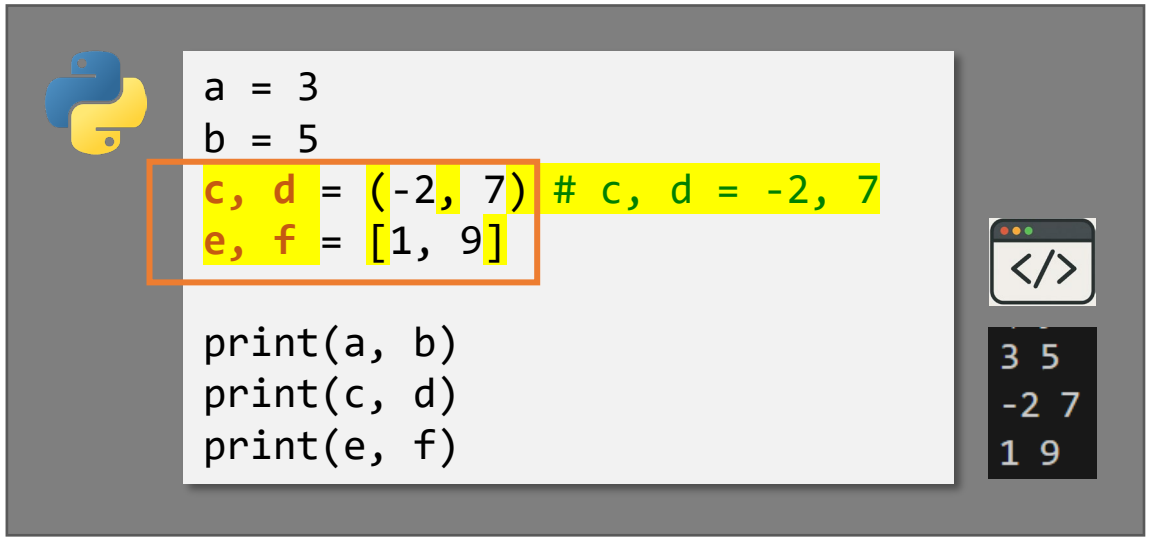
```
c =  1    # int
d =  1,   # tuple
e =  (1,) # tuple
f =  (1)  # int
print(type(d))
print(type(e))
print(type(f))ZZZZZ
```

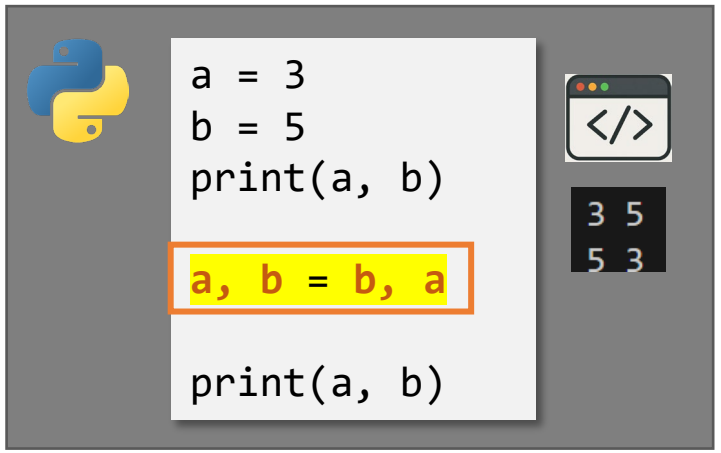
```
<class 'tuple'>
<class 'tuple'>
<class 'int'>
```

```
g = (1 2) # error
```

```
    g =  (1 2) # cause error
           ^^^
SyntaxError: invalid syntax. Perhaps you forgot a comma?
```

Assignment of multiple variables





Set Type

- Unordered
- Unique
- Mutable

type

```
a = {1, 'hello'}
b  = set([1, 'hello'])
```

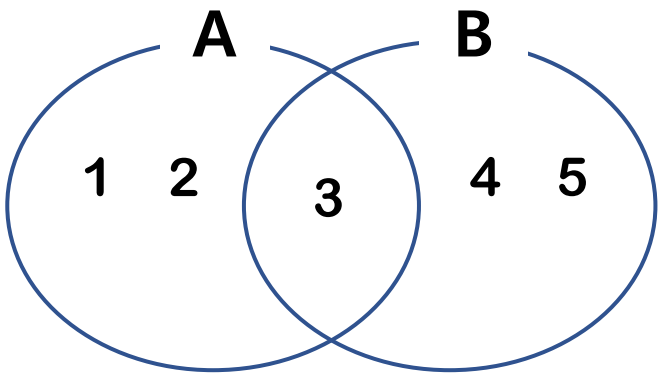
```
c = {1, -5, 8, 8}
print(c)
```

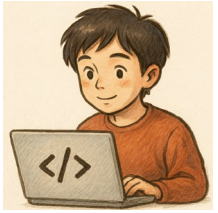
```
{8, 1, -5}
```

set

```python
A = {1, 2, 3}
B = {3, 4, 5}

print(A | B)   # Union: {1, 2, 3, 4, 5}
print(A & B)   # Intersection: {3}
print(A - B)   # Difference: {1, 2}
print(A ^ B)   # Symmetric difference: {1, 2, 4, 5}
```

```
{1, 2, 3, 4, 5}
{3}
{1, 2}
{1, 2, 4, 5}
```





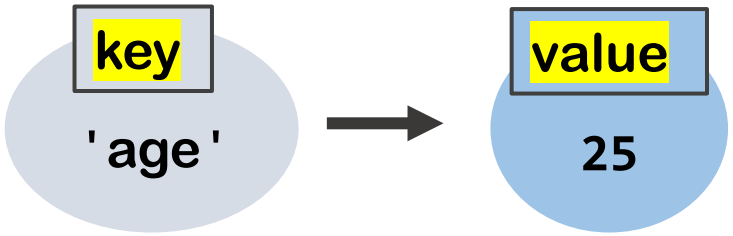
Dictionary Type

```
a = {'age': 25}
print(a)
a['age'] = 30
print(a)
```

mutable type

```
{'age': 25}
{'age': 30}
```

- "Keys" must be unique.
- "Values" can be duplicated



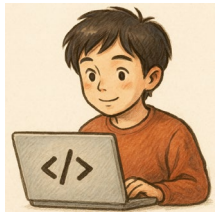
dictionary

```
b = {'age': 25, 'myList':[2,3,4]}

print(b)
print(b['age'])
print(b['myList’])
print(type(b))
```

```
{'age': 25, 'myList': [2, 3, 4]}
25
[2, 3, 4]
<class 'dict'>
```

```
c = dict(age = 25, myList = [2,3,4])
print(c)
```

```
{'age': 25, 'myList': [2, 3, 4]}
```



if statement

```
age = 30

if age > 40:
    print('old')
else:
    print('young')
```

```
young
```

while loop

```
count = 0

while count <= 5:
    print(count)
    count += 2
```

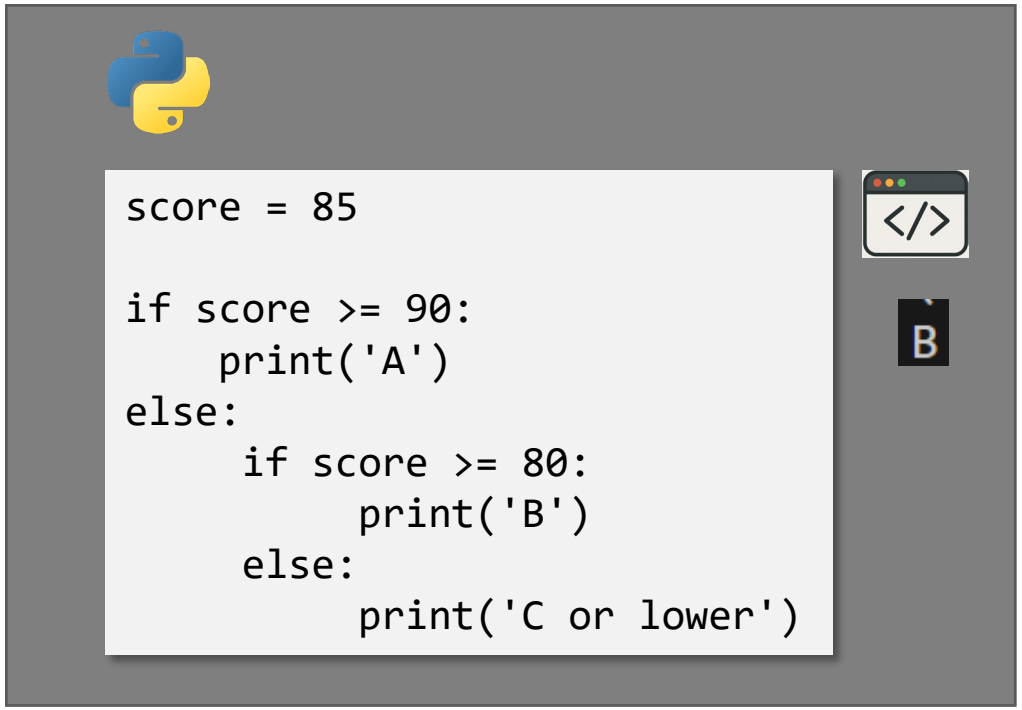
```
0
2
4
```

for loop

```
myList = [2, -4, 30]

for i in myList:
    print(i)
```

```
2
-4
30
```

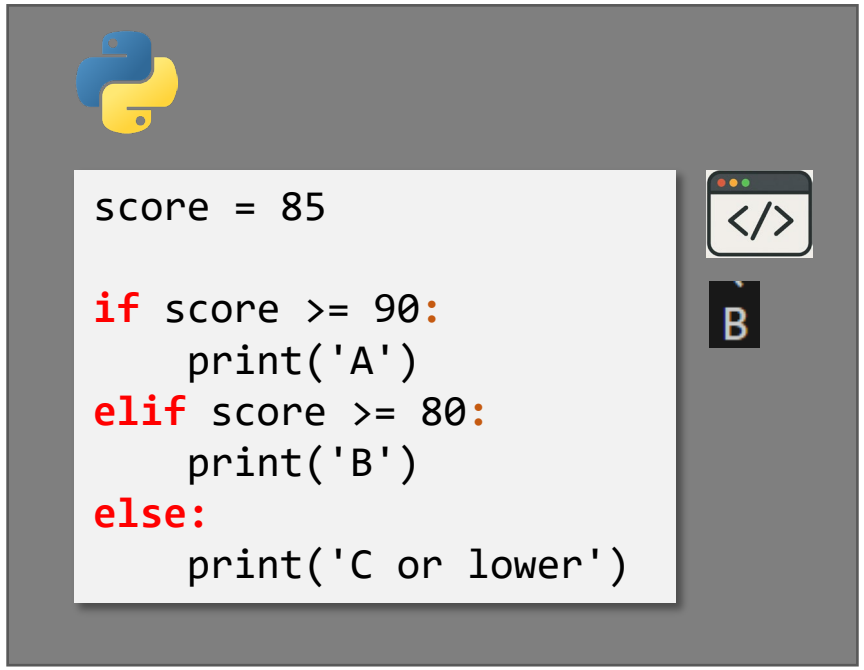
if statement



```
score = 85

if score >= 90:
    print('A')
else:
     if score >= 80:
          print('B')
     else:
          print('C or lower')
```

B

=



```
score = 85

if score >= 90:
    print('A')
elif score >= 80:
    print('B')
else:
    print('C or lower')
```

B

if statement

```
x = 15

if x > 10:
    if x < 20:    # nested if
        print("x is between 10 and 20")

if 10 < x < 20:
    print("x is between 10 and 20")
```

```
x is between 10 and 20
x is between 10 and 20
```

```
menu = ('milk', 'egg')
choice = 'egg'

if choice in menu:
     print('OK')
else:
     print('Unavailable')
```

```
OK
```

while loop

```
count = 0

while count <= 5:
    print(count)
    count += 2
```

```
0
2
4
```

```
n = 1

while True:  # infinite loop
    print(n)
    if n == 3:
        break  # stop the loop
    n += 1
```

```
1
2
3
```

```
n = 1

while n < 5:
    n += 1
    if n == 3:
        continue  # skip the rest of the loop for this iteration
    print(n)
```

```
2
4
5
```

while loop

```
while n < 5:
    print(n)
    if n == 3:
        break  # stop the loop
    n += 1
else:
    print('loop finished')
```

```
1
2
3
```

```
while n < 5:
    print(n)
    n += 1
else:
    print('loop finished (no break)')
```

```
1
2
3
4
loop finished (no break)
```

for loop

```
myList = [2, -4, 30]

for i in myList:
    print(i)
```

```
2
-4
30
```

```
a = [(1,2), (3,4), (5,6)]

for (first, last) in a:
    print(first, last)
```

```
1 2
3 4
5 6
```

```
for i in range(4):
    print(i)
```

```
0
1
2
3
```

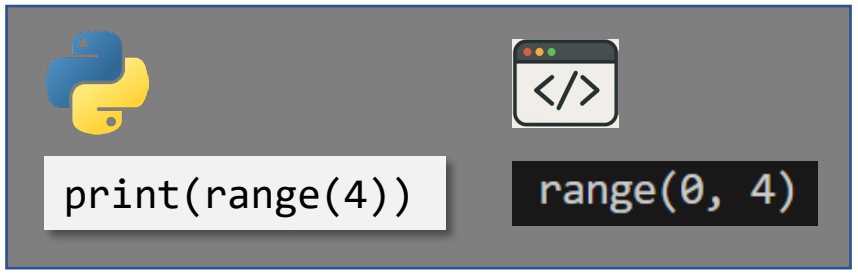
```
for i in range(2,5):
    print(i)
```

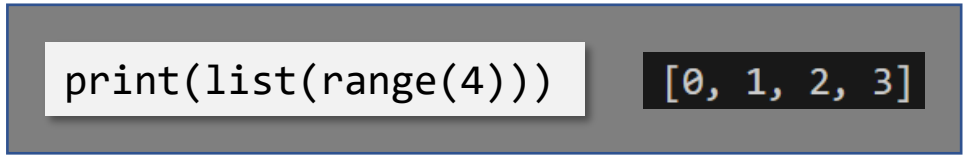
```
2
3
4
```

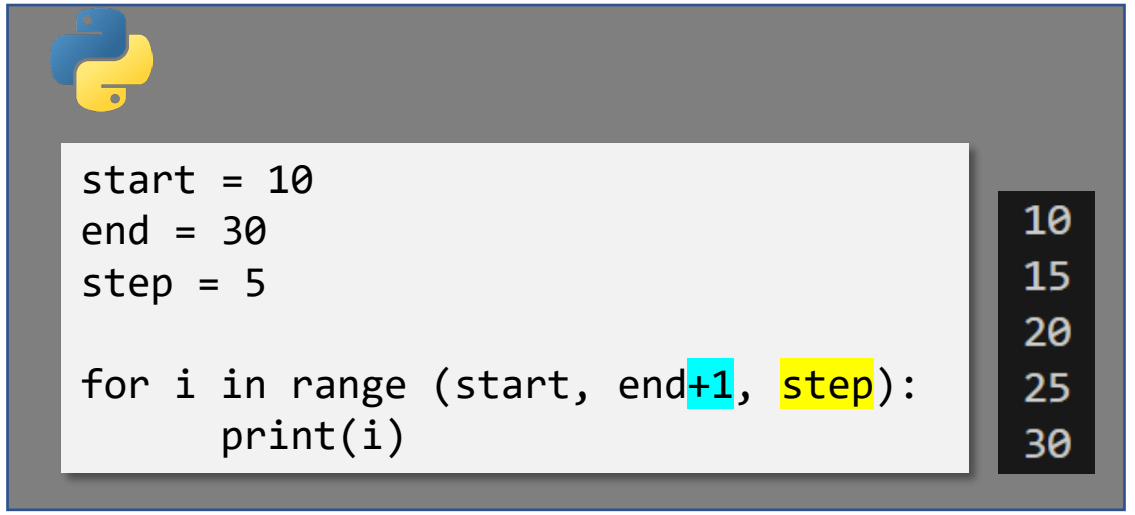
range(..)

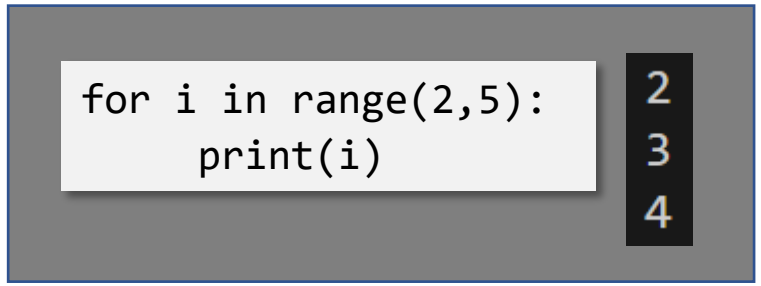
- **range**() is a function that generates a sequence of numbers in Python.

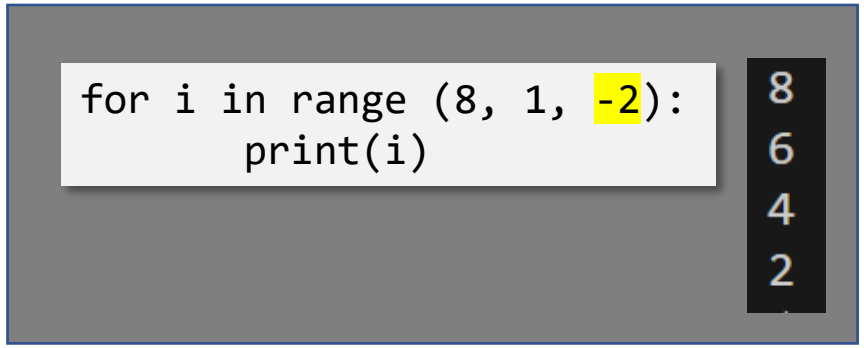
- **range() is not a list, but a lazy sequence object.**











Break and **continue** in **for** loop

```
for i in range(1, 5):
    if i == 3:
        print("Breaking the loop at i =", i)
        break   # completely exits the loop
print("Current value:", i)
```

```
Current value: 1
Current value: 2
Breaking the loop at i = 3
```

```
for i in range(1, 5):
    if i == 3:
        print("Skipping this iteration at i =", i)
        continue   # skip the rest of the loop
    print("Current value:", i)
```

```
Current value: 1
Current value: 2
Skipping this iteration at i = 3
Current value: 4
```

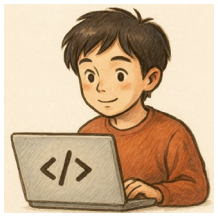
Functions

```
def myfunc():
    print('hello')

myfunc()
```

```
hello
```

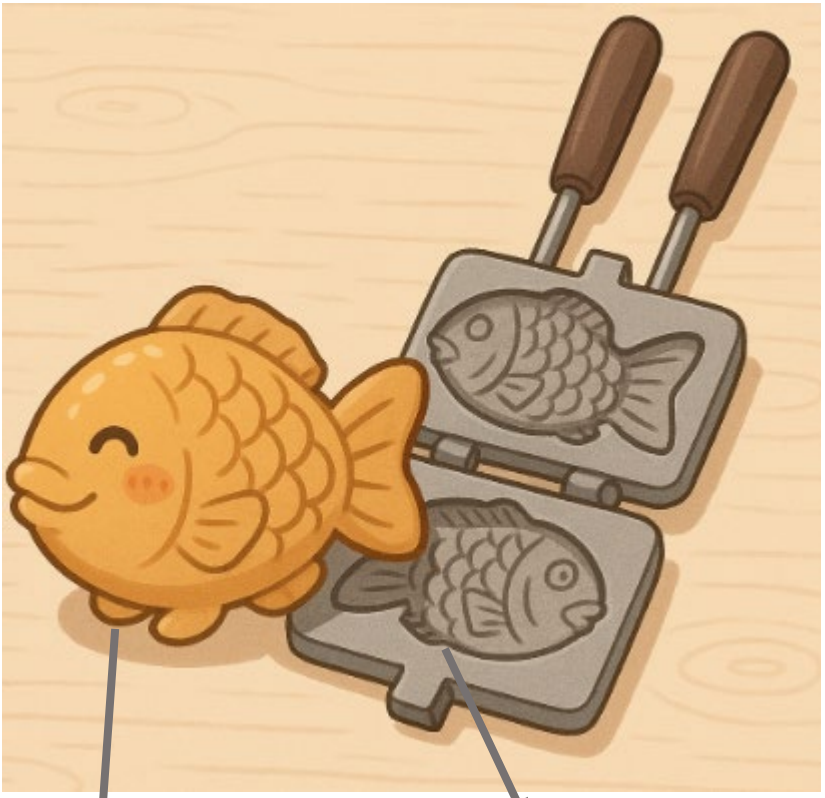
```
def add(x, y):
    return x+y

print(add(2,5))
```

```
7
```



Class

```
class Fish:
    pass

myFish = Fish()
```



myFish

(instance of class)

(=**object**)

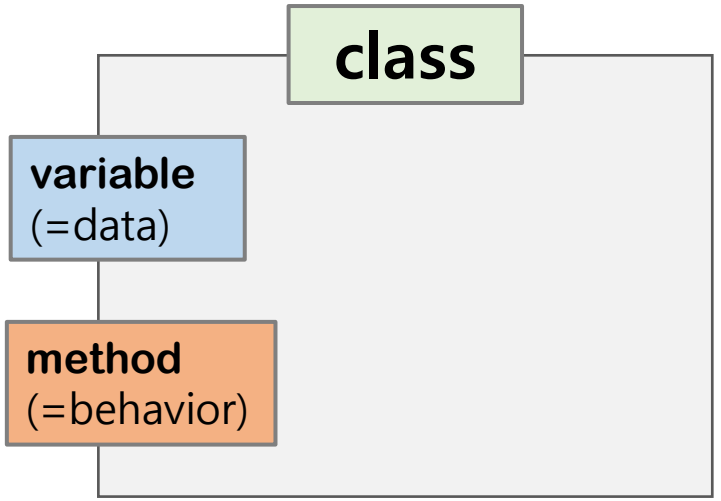
Fish

(class name)

Variables and **Methods** in class

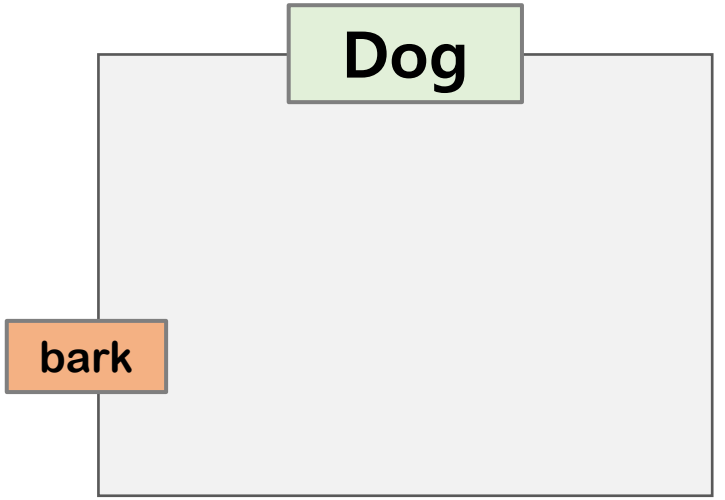
```python
class Dog:
    def bark(self):        # method
        print('mung mung')

my_dog = Dog()
my_dog.bark()
```

```
mung mung
```



In Python, an **object** is an **instance of a class** that contains both **variables (data)** and **methods (behaviors)**.

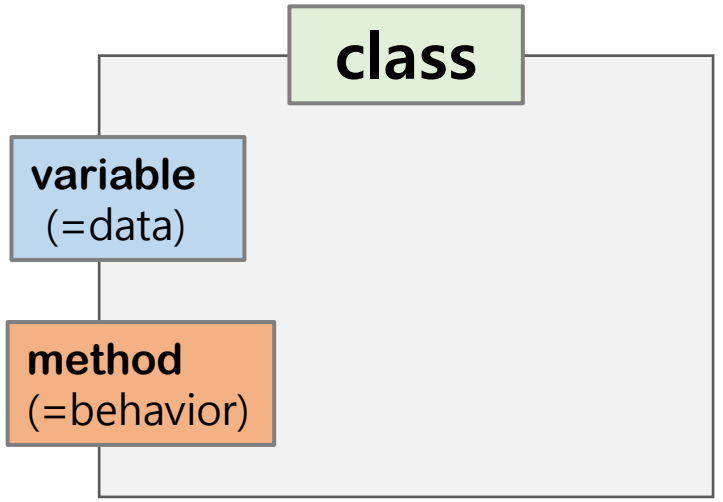
- **variables**: Variables that belong to the object.
- **methods**: Functions that belong to the object and can operate on its data.

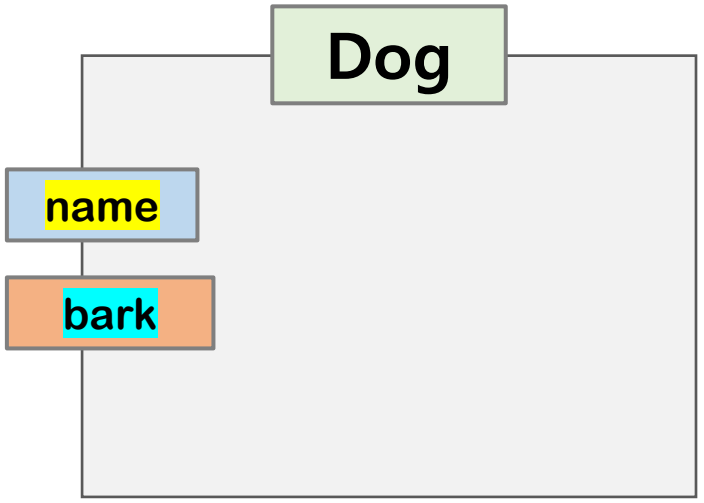


__init__(self, ..)

```python
class Dog:
  def __init__(self, dog_name): # initialize
     self.name = dog_name    # instance variable

  def bark(self):              # method
     print(f'{self.name}: mung mung')

my_dog = Dog('kongi')
my_dog.bark()

your_dog = Dog('baduki')
your_dog.bark()
```

```
kongi: mung mung
baduki: mung mung
```





Exceptions

```
try:
    4/0
except:
    print('error')
```

```
error
```

```
try:
    4/0
except Exception as e:
    print(e)
else:
    print(no Exception error)
finally:
    print('finally')
```

```
division by zero
finally
```

```
try:
    pass
except Exception as e:
    print(e)
else:
    print(no Exception error)
finally:
    print('finally')
```

```
no Exception error
finally
```

User Input/Output (1)

Input( ) function

```
a = input()
print(a + ' world')
```

```
hello
hello world
```

```
a = input('Please enter your name: ')
print('Hi ' + a)
```

```
Please enter your name: chulsoo
Hi chulsoo
```

User Input/Output (2)

Open( ) function (1)

The **open( )** function is used to **open a file** in Python. It **returns a file object**, which allows you to read from or write to the file.

open(file, mode='r', encoding=None)

```
mode: The mode in which the file is opened. Some common modes are:

'r' → Read (default). File must exist.
'w' → Write. Creates a new file or overwrites existing content.
'a' → Append. Adds data to the end of the file.
```

```
encoding: Encoding type (e.g., "utf-8") when working with text files.
```

Reading a file

```
file = open("example.txt", "r", encoding="utf-8")
content = file.read() # Read the whole content
print(content)

file.close() # Always close the file
```

User Input/Output (3)

Open( ) function (2)

Writing a file

```
file = open("example.txt", "w", encoding="utf-8")
content = file.write("Hello world") # write some text to the file
file.close() # Always close the file
```

```
with open("example.txt", "w", encoding="utf-8") as file:
    content = file.write("Hello world")  # write some text to the file
```

- When the program leaves the with block, file.close() is called automatically.
- This makes the code safer and cleaner.

User Input/Output (4)

Open( ) function (3)

Example.txt

```
Hello world
Hi there !
```

```
with open("example.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line)
```

```
Hello world

Hi there !
```

Python

Intermediate level skills

Modules

Module = a signle Python file

Packages

Package = a folder that contains multiple modules

Modules and Packages

Module = a signle Python file

Package = a folder that contains multiple modules

For Example

```
project/
├── main.py        # module. entry point (= main script)
├── say.py         # module
│
└── myPackage/     # package (folder)
    ├── __init__.py # a special Python file that tells Python "this folder is a package."(< Python 3.2) and performs initialization
    ├── calc.py    # module
```

How to use functions in other modules

```
project/
├── main.py
├── say.py
│
└── myPackage/
    └── calc.py
```

say.py

```
def hello():
    print('hello world')
```

myPackage

calc.py

```
def add (x, y):
    return x + y
```

main.py

```
import say
say.hello()  # hello world

import myPackage
print(myPackage.calc.add(2,3)) # error because there is no __init__.py
```

```
hello world
    print(myPackage.calc.add(2,3))
AttributeError: module 'myPackage' has no attribute 'calc'
```

How to use functions in other modules

```
project/
├── main.py
├── say.py
│
└── myPackage/
    ├── __init__.py
    ├── calc.py
```

say.py

```
def hello():
    print('hello world')
```

myPackage

__init__.py

```
from . import calc
```

calc.py

```
def add (x, y):
    return x + y
```

main.py

```
import say
say.hello()  # hello world

import myPackage
print(myPackage.calc.add(2,3)) # OK
```

```
hello world
5
```

=

main.py

```
import say
say.hello()  # hello world

from myPackage.calc import add
print(add(2,3)) # OK
```

```
hello world
5
```

__name__ == "__main__"

say.py

```
def hello():
    print('hello world')

if __name__ == "__main__":
    print("my_module.py is running directly")
```

```
my_module.py is running directly
```

world.py

```
import say
say.hello()  # hello world
```

```
hello world
```

String Formatting

```
# f formatting (after Python 3.6)

name = kongi
age = 5
X = f'{name} is {age} years old.'

print(X)
```

```
kongi is 5 years old.
```

str **Formatting (1)**

```
a = 'I have %d apples.' % 4
print(a)
```

```
I have 4 apples.
```

% d # OK

```
b = 'The dog weighs %f kg.' % 5.2
print(b)
```

```
The dog weighs 5.2 kg.
```

```
d = 'I have %d %s' % (4, 'apples.')
print(d)
```

```
I have 4 apples.
```

str formatting **Method** Call (2)

```
result = '{} {} weigh {} grams.'.format(10,'eggs', 605.2)
print(result)
```

```
10 eggs weigh 605.2 grams.
```

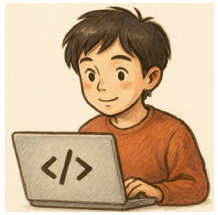
```
result = '{0} {1} weigh {2} grams.'.format(10,'eggs', 605.2)
print(result)
```

```
10 eggs weigh 605.2 grams.
```

```
X = '{0} {food}'.format(9, food='apples')
print(X)
```

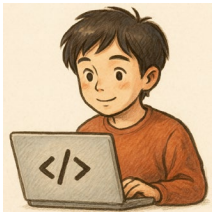
```
9 apples
```

f str Formatting (after python 3.6)



```
name = 'kongi'
age = 5
X = f'{name} is {age} years old.'
print(X)
```

```
kongi is 5 years old.
```



Methods of lists, strings, sets, and dictionaries

List Methods (1)

```python
a = [1, 2, 3]
b = a
a.append(4)   # in-place operation
print(a)  # [1, 2, 3, 4]
print(b)  # [1, 2, 3, 4] -> b is also changed since it references the same object
```

```
[1, 2, 3, 4]
[1, 2, 3, 4]
```

In Python, **in-place** operation modifies the existing object directly.

- ✓ **In-place** = modifies the existing object
- ✓ Non in-place = creates and returns a new object

List Methods (2)

Non In-place

```
a = [1, 2]
b = a
a = a + [3]    # non in-place: creates and returns a new object
print(a)  # [1, 2, 3]
print(b)  # [1, 2] -> variable a points to a new object
```

```
[1, 2, 3]
[1, 2]
```

In-place

```
a = [1, 2]
b = a
a += [3]   # in-place
print(a)  # [1, 2, 3]
print(b)  # [1, 2, 3] -> the original object was modified
```

```
[1, 2, 3]
[1, 2]
```

List Methods (3)

```
# print(num)
```

```
num = [1, 2, 3]
num.extend([4, 5])    # [1, 2, 3, 4, 5]
print(num)
```

```
[1, 2, 3, 4, 5]
```

```
num = [7, 8, 9, 10]
num.insert(1, 50) # [7, 50, 8, 9, 10]
```

```
num = [7, 8, 9, 10]
a = num.pop()    # num == [7, 8, 9], a == 10
```

List Methods (4)

```
num = [7, 8, 9, 10]
num.remove(9)  # num == [7, 8, 10]
print(num)
```

```
[7, 8, 10]
```

```
num = ['cat', 'dog', 'horse']
a = num.index('cat')  # a == 0
print(a)
```

```
0
```

```
num = [1, 4, 3, 2]
num.sort()   # [1, 2, 3, 4]
print(num)
```

```
[1, 2, 3, 4]
```

```
num = [7, 8, 9, 10]
num.reverse()  # [10, 9, 8, 7]
print(num)
```

```
[10, 9, 8, 7]
```

del

- ✓ **del** is not a function, but **a Python keyword**.
- ✓ That means it's a language construct, not something like print( ) or len( ).
- ✓ **del** can delete not only list elements but also whole variables

```
num = [7, 8, 9, 10]
del num[2:]   # [7, 8]
print(num)
```

```
[7, 8]
```

```
x = 100
del x
print(x) # Error: x was deleted
```

```
    print(x)
NameError: name 'x' is not defined
```

```
x = (2, 4, 5)
del x
print(x) # Error: x was deleted
```

```
    print(x)
NameError: name 'x' is not defined
```

String Methods (1)

```
a = 'hello'
print(a.find('e'))
```

```
1
```

```
a = ','.join('abcd')
print(a)  # a, b, c, d
```

```
a,b,c,d
```

```
a = 'HELLO'    # string is immutable
b = a.lower() # non in-place
print(a)
print(b)
```

```
HELLO
hello
```

String Methods (2)

```
a = 'hello world'
b = a.split()
print(a)
print(b)
```

```
hello world
['hello', 'world']
```

```
c = 'a:b:c:d'
d = c.split(':')
print(c)
print(d)
```

```
a:b:c:d
['a', 'b', 'c', 'd']
```

```
a = 'hello world'
b = a.replace('hello', 'hi')
print(a)
print(b)
```

```
hello world
hi world
```

Set Methods

```
s = {7, 8, 9}
s.add(50)      # {7, 8, 9, 50}
```

```
s = {7, 8, 9}
s.update([-10, 3])   # {3, 7, 8, 9, -10}
```

```
s = {7, 8, 9}
s.remove(8)    # {9, 7}
```

```
A = set([1, 2, 3, 4])  # {1, 2, 3, 4}
B = set([3, 4, 9, 20]) # {3, 4, 9, 20}
A & B     # {3, 4}
A | B     # {1, 2, 3, 4, 9, 20}
A - B     # {1, 2}
```

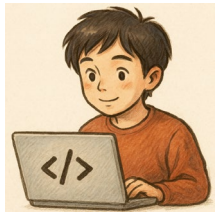
Dictionary Methods

```
d = {'name': 'chulsoo', 'age': 21 }
print(d.keys())
print(d.values())
print(d.items())
print(d.clear())
```

```
dict_keys(['name', 'age'])
dict_values(['chulsoo', 21])
dict_items([('name', 'chulsoo'), ('age', 21)])
None
```

```
d = {'name': 'chulsoo', 'age': 21 }
a = 'name' in d
b = 'weight' in d
print(a, b)
```

```
True False
```



Copy List

```python
a = [1, 2, 3]
b = a

print(id(a), id(b))
print (a is b)

a[0] = 8
print(a, b)
```

```
2324562958528 2324562958528
True
[8, 2, 3] [8, 2, 3]
```

```python
from copy import copy

c = [1, 2, 3]
d = c[:] # d = c.copy()

print(id(c), id(d))
print (c is d)

c[0] = 8
print(c, d)
```

```
2277206448320 2277206872320
False
[8, 2, 3] [1, 2, 3]
```

✓ Variables, **a** and **b** do **not** create two separate lists.
✓ They both **reference (or point to) the same list object** in memory, so they share the same underlying data."

Shallow Copy vs Deep Copy

(1) Shallow Copy

```python
import copy

original = [[1, 2, 3], [4, 5, 6]] # Original list with nested list

shallow_copied = copy.copy(original) # Shallow copy: the same as shallow_copy = original
shallow_copied[0][0] = 99 # Modify nested list inside the shallow copy

print("Original:", original)
print("Shallow Copy:", shallow_copied)
```

```
Original: [[99, 2, 3], [4, 5, 6]]
Shallow Copy: [[99, 2, 3], [4, 5, 6]]
```

- ✓ A shallow copy creates a new object, but does not recursively copy nested objects.
- ✓ Instead, it just copies references of nested objects.
- ✓ This means that if the original object contains lists (or other mutable objects) inside, both the original and the copied object will point to the same inner objects.

Shallow Copy vs Deep Copy

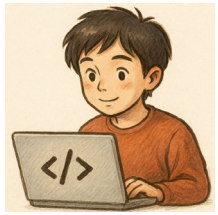
(2) Deep Copy

```python
import copy

original = [[1, 2, 3], [4, 5, 6]] # Original list with nested list

# Deep copy: Creates a completely independent clone, including all nested objects.
deep_copied = copy.deepcopy(original)
deep_copied[0][0] = 99 # Modify nested list inside the deep copy

print("Original:", original)
print("Deep Copy:", deep_copied)
```

```
Original: [[1, 2, 3], [4, 5, 6]]
Deep Copy: [[99, 2, 3], [4, 5, 6]]
```

✓ A deep copy creates a completely independent copy of the object, including all nested objects.

✓ This means that modifications in the deep copy will not affect the original object.



Mutable vs Immutable Object Types (1)

In Python, the difference between mutable and immutable objects mainly shows up when you pass them into a function.

(1) Immutable objects (like int, float, str, tuple)

cannot be changed in place. If you try to change them, Python creates a new object.

```
def add_one(x):
    x = x + 1   # This creates a NEW int object
    print("Inside function:", x)

num = 10              # int: immutable object
add_one(num)
print("Outside function:", num)
```

```
Inside function: 11
Outside function: 10
```

Mutable vs Immutable Object Types (2)

(2) Mutable objects (like list, dict, set)

can be changed in place, so modifications inside a function will affect the original object even after the function ends.

```
def add_item(my_list):
    my_list.append(100)   # Modify in place
    print("Inside function:", my_list)

nums = [1, 2, 3]  # list: mutable object
add_item(nums)
print("Outside function:", nums)
```

```
Inside function: [1, 2, 3, 100]
Outside function: [1, 2, 3, 100]
```

Arguments of Function (1)

(1) Positional or keyworkd arguments

```
def f(a, b):
    print(a, b)

f(1, 2)       # positional
f(a=1, b=2)   # keyword
```

```
1 2
1 2
```

(2) Arbitrary arguments

```
def f(a, *args):          #*args: arbitray
    print(a, args)

f(1, 2, 3, 4)   # a=1, args=(2,3,4)
```

```
1 (2, 3, 4)
```

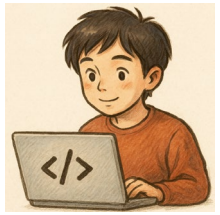
The parameter *args collects all arguments into a tuple.

Arguments of Function (2)

(3) Order of arguments: f(normal, *arbitray, **keyword)

```
def f(x, y, *args, **kwargs):
    print(args, x, y, kwargs)

f(1, 2, 3, 'hello', age=20)
```

```
(3, 'hello') 1 2 {'age': 20}
```



class

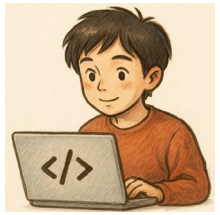
(1) Simplest class

```
class Dog:
    pass

my_dog = Dog()
```

(2) Initialization

```
class Dog:
    def __init__(self):
        print('initialization')

my_dog = Dog()
```

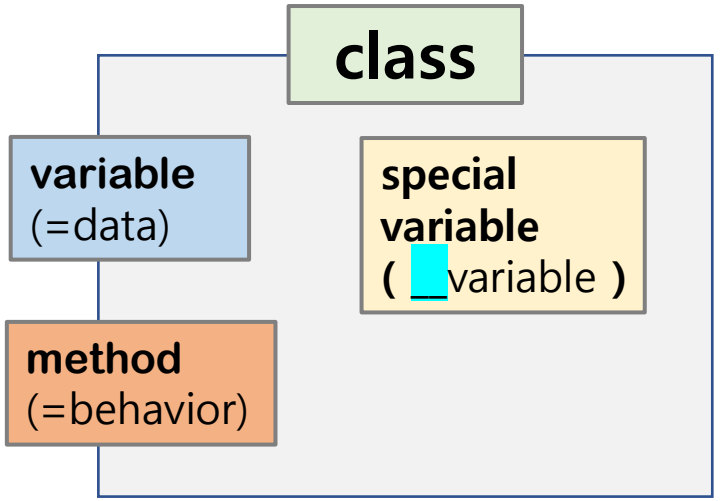
```
initialization
```

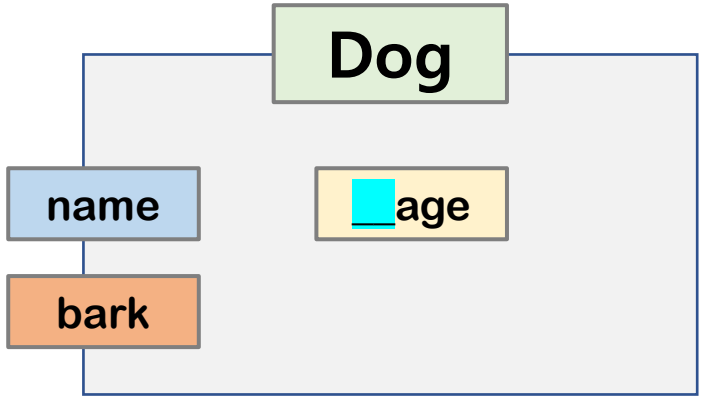


class Variables and Methods

```
class Dog:
    def __init__(self, dog_name):
        self.name = dog_name      # instance variable
        self.__age = 5            # special variable

    def bark(self):               # method
        print(f'{self.name} ({self.__age}): mung mung')

my_dog = Dog('kongi')
my_dog.bark()
```

```
kongi (5): mung mung
```






```
my_dog.name = 'baduki'
my_dog.__age = 10
my_dog.bark()
```

```
baduki (5): mung mung
```

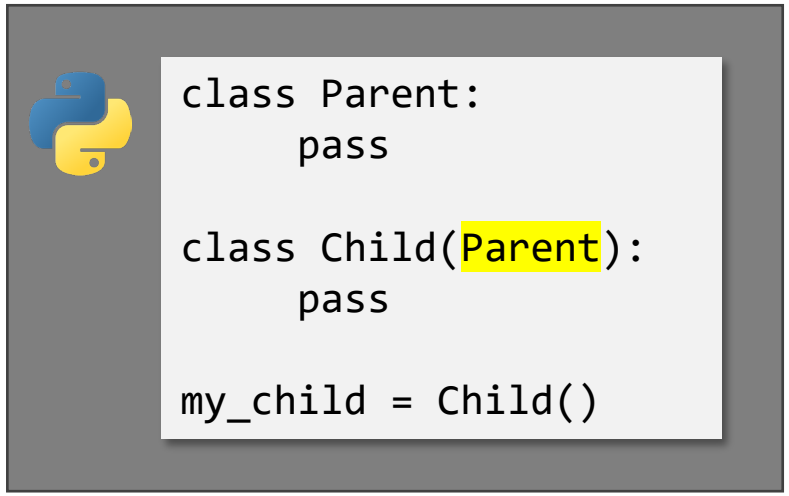
`self.__age = 5` (internally stored as `self._Dog__age` due to name mangling

Class Variables

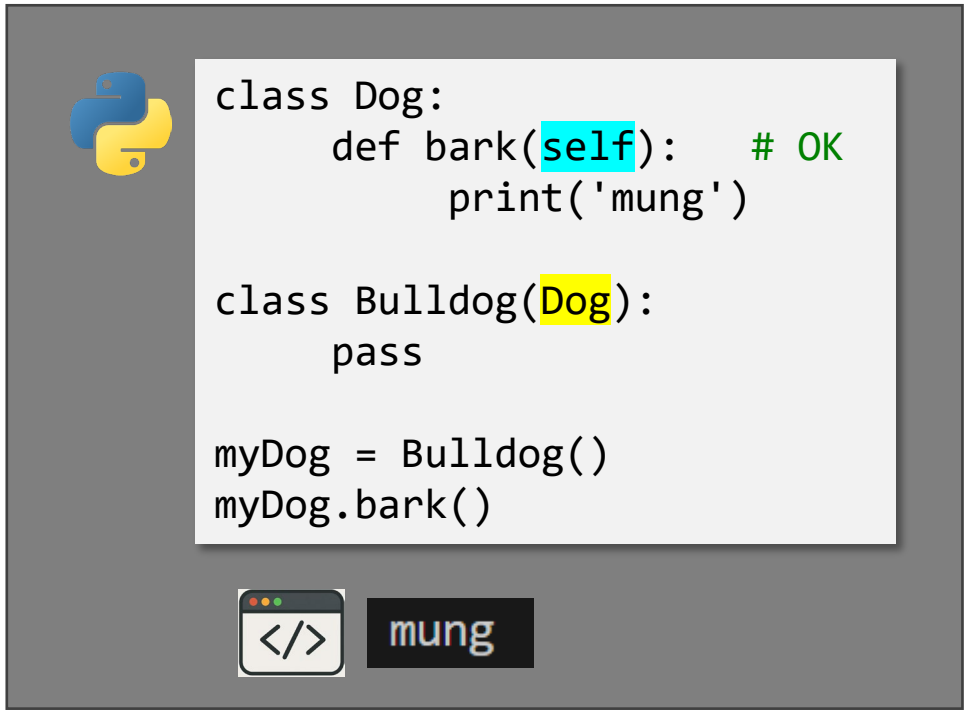
```
class Dog:
     num_leg = 4                    # class variable (shared class data)

     def __init__(self, dog_name):
          self.name = dog_name      # instance variable (per-instance data)

my_dog = Dog('kongi')
your_dog = Dog('baduki')

print(my_dog.num_leg)
print(Dog.num_leg)
```

```
4
4
```

Class Inheritance (1)



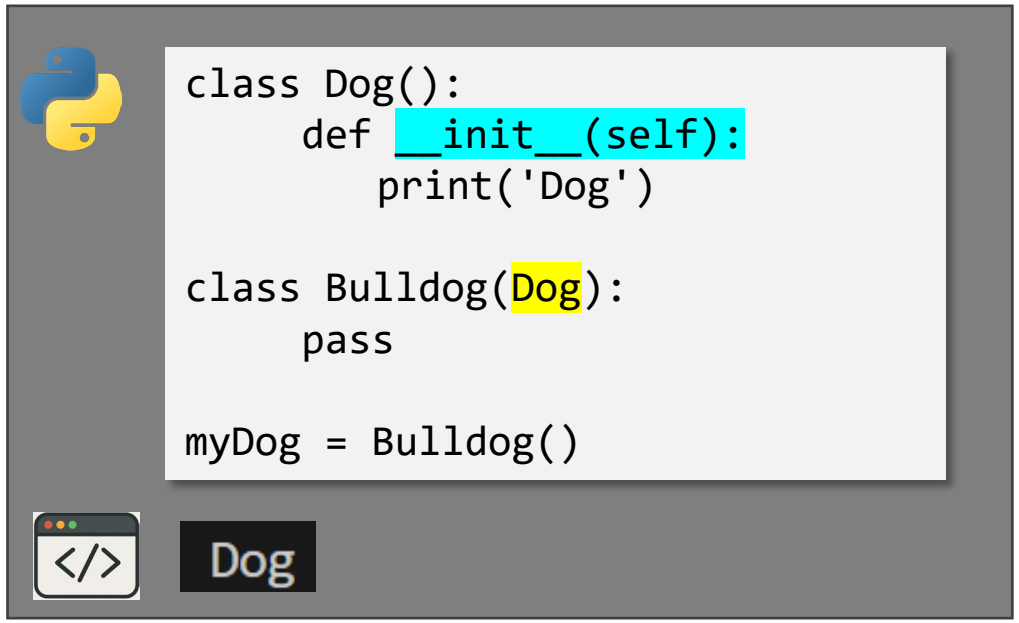
```
class Parent:
    pass

class Child(Parent):
    pass

my_child = Child()
```

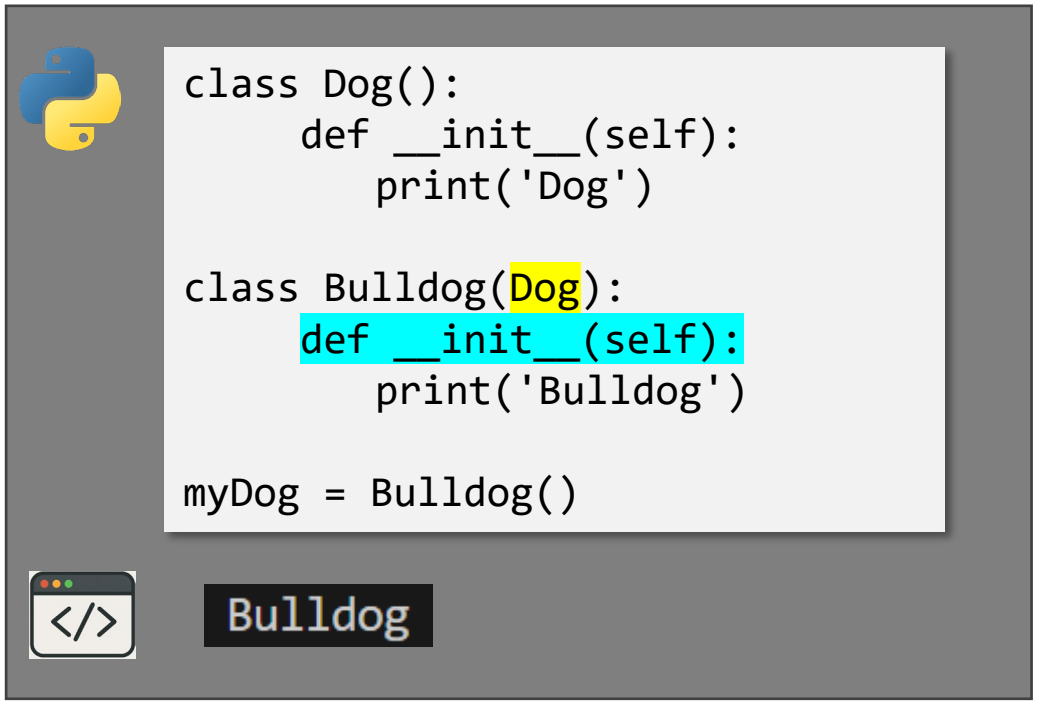


```
class Dog:
    def bark(self):   # OK
        print('mung')

class Bulldog(Dog):
    pass

myDog = Bulldog()
myDog.bark()
```

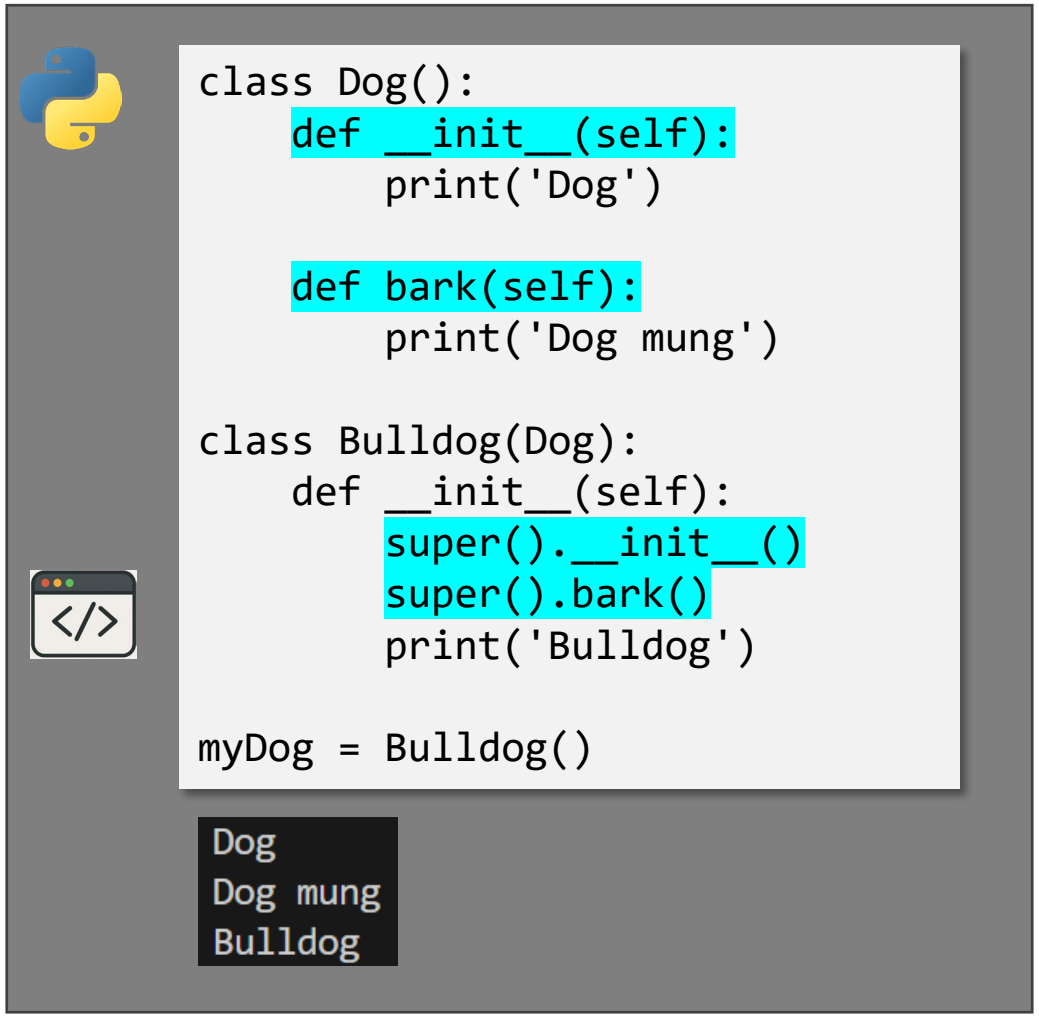
```
mung
```

Class Inheritance (2)





Class Inheritance (3)



```
class Dog():
    def __init__(self):
        print('Dog')

    def bark(self):
        print('Dog mung')

class Bulldog(Dog):
    def __init__(self):
        super().__init__()
        super().bark()
        print('Bulldog')

myDog = Bulldog()
```

```
Dog
Dog mung
Bulldog
```

Method Overriding

```
class Dog:
      def bark(self):
            print('mung')

class Bulldog(Dog):
     def bark(self):
            print('kau wa woo')

myDog = Bulldog()
myDog.bark()
```

```
kau wa woo
```

```
class Dog:
      def bark(self):
            print('mung')

class Bulldog(Dog):
     def bark(self):
            super().bark()
            print('kau wa woo')

myDog = Bulldog()
myDog.bark()
```

```
mung
kau wa woo
```

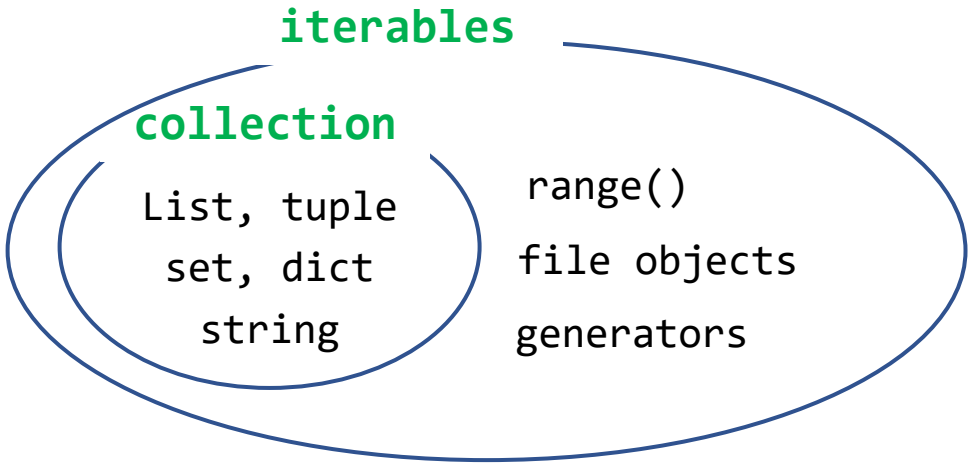
Python

Advanced level skills

Iterable objects

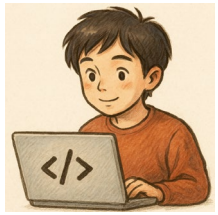
```
for i in [2, -4, 30] :
    print(i)
```

```
2
-4
30
```




```
for element in iterable
```

- An **iterable** object is any object that can be used in a for ... in ... loop.
  (= an **object that can be iterated over**).
- All collections (list, tuple, set, dict, string) are iterables.
- Other objects like range( ), generators, and file objects are also iterables.
  👉 Therefore, **iterable ⊃ collection** (iterables form a broader concept).



Iterator

```python
a = [1, 2, 3]
it = iter(a)   # create an iterator from the list

print(next(it))  # 1
print(next(it))  # 2
print(next(it))  # 3
print(next(it))  # raises StopIteration (no more elements)
```

```
1
2
3
Traceback (most recent call last):
  File "D:\dev\python\myTorch\pytorch\pytorch_test\myTest\simple01.py", line 43, in <module>
    print(next(it))
StopIteration
```

An **iterable** is like a "container" (it can give us an iterator).

An **iterator** is the "machine" that pulls out items from that container one by one.

Custom Iterator

An iterator is an object that implements two special methods:

- `__iter__()` → returns the iterator object itself
- `__next__()` → returns the next value, or raises StopIteration when there are no more items

```
class CountDown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self  # the iterator object returns itself

    def __next__(self):
        if self.current <= 0:
            raise StopIteration  # no more values
        num = self.current
        self.current -= 1
        return num

for number in CountDown(5):   # Using the iterator
    print(number)
```

```
5
4
3
2
1
```

Generator

- A generator is a simpler way to create an iterator using the yield keyword.
- When a function contains yield, it automatically becomes a generator function, returning a generator object.
- Generators **automatically implement** `__iter__()` and `__next__()` for us.

```
def countdown(start):
    while start > 0:
        yield start  # pauses here and returns value
        start -= 1

# Using the generator
for num in countdown(5):
    print(num)
```

```
5
4
3
2
1
```

Bulit-in Functions (1)

enumerate() function

```
animals = ["dog", "cat", "cow"]

# Using enumerate without specifying start
for index, animal in enumerate(animals):
    print(index, animal)

print("-----")

# Using enumerate with a custom start value
for index, animal in enumerate(animals, start=100):
    print(index, animal)
```

```
0 dog
1 cat
2 cow
-----
100 dog
101 cat
102 cow
```

Enumerate() returns pairs of (index, element) that we can use in a loop.

Bulit-in Functions (2)

zip() function (1)

(1) Basic example

```
animals = ["dog", "cat", "cow"]
ages = [1, 2, 5]

zipped = zip(animals, ages)
print(list(zipped))
```

```
[('dog', 1), ('cat', 2), ('cow', 5)]
```

Bulit-in Functions (3)

zip() function (2)

(2) Iterating with zip()

```
animals = ["dog", "cat", "cow"]
ages = [1, 2, 5]

for animal, age in zip(animals, ages):
    print(f"{animal} is {age} years old.")
```

```
dog is 1 years old.
cat is 2 years old.
cow is 5 years old.
```

(3) Unzipping: reverse of zip()

we can use * to unzip back into separate lists.

```
pairs = [('dog', 1), ('cat', 2), ('cow', 5)]
animals, ages = zip(*pairs)

print(animals)  # ("dog", "cat", "cow")
print(ages)   # (1, 2, 5)
```

```
('dog', 'cat', 'cow')
(1, 2, 5)
```

Bulit-in Functions (4)

filter() function

filter(function, iterable) applies a function to each element of an iterable (like a list or tuple)

- It returns only the elements for which the function returns True.
- The result is an iterator (we need to wrap it in list( ) to see the values).

```
num = [1, 2, 3, 4, 5, 6]

# extract only even numbers
def is_even(n):
    return n % 2 == 0

result = filter(is_even, num)
print(list(result))
```

```
[2, 4, 6]
```

Lambda Function (1)

```
# normal function
def add(x, y):
    return x + y

# Equivalent lambda function
add_lambda = lambda x, y: x + y

print(add(3, 5))        # 8
print(add_lambda(3, 5)) # 8
```

```
8
8
```

- Lambda functions are good for **short, throwaway functions**.
- They should not be used if the logic is complex (for readability).

Lambda Function (2)

(1) Using Lambda with map()

```
num = [1, 2, 3, 4, 5]

# Square each number
squares = list(map(lambda x: x**2, num))
print(squares)
```

```
[1, 4, 9, 16, 25]
```

(2) Using Lambda with filter()

```
num = [10, 15, 20, 25, 30]

# extract only even numbers
evens = list(filter(lambda x: x % 2 == 0, num))
print(evens)
```

```
[10, 20, 30]
```

List Comprehension

```
a = [x**2 for x in range(1_000_000)]
print(a[0:5])
```

```
[0, 1, 4, 9, 16]
```

```
b = [1, 2, 3, 4]
results = [num*3 for num in b if num%2 == 0]
print(b)
```

```
[1, 2, 3, 4]
```

List comprehension expression (1)

```
M = [[1,2,3],
     [4,5,6],
     [7,8,9]]

col_0 = [row[0] for row in M]
```

```
print(col_0) # column 0 of M
print(M)     # unchanged
```

```
[1, 4, 7]
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
diag = [M[i][i] for i in range(3)]  # range(3) == [0,1,2]
print(diag)
```

```
[1, 5, 9]
```

Generator

```
M = [[1,2,3],
     [4,5,6],
     [7,8,9]]

G = (row[0] for row in M)  # generator
```

```
print(G)
print(next(G))
print(next(G))
print(next(G))
```

```
<generator object <genexpr> at 0x000001CDC84D2180>
6
15
24
```

Closure (1)

Closure is a function that "**remembers**" the variables from the scope in which it was created, even if that scope is no longer active.

```python
def outer_function(msg):
    def inner_function():
        print("Message:", msg)
    return inner_function  # returning the inner function

# Create closures
hello_func = outer_function("Hello")
bye_func = outer_function("Goodbye")

# Even though outer_function has finished execution,
# the inner functions remember the 'msg' variable
hello_func()  # Output: Message: Hello
bye_func()    # Output: Message: Goodbye
```

```
Message: Hello
Message: Goodbye
```

Closure (2)

```python
def my_decorator(func):
    def wrapper():
        print("Before func()")
        func()
        print("After func()")
    return wrapper  # return the closure

def say_hello():
    print("Hello!")

decorated_func = my_decorator(say_hello)

# decorated_func is actually 'wrapper'
# which remembers 'func' decorated_func()
decorated_func()
```

```
Before func()
Hello!
After func()
```

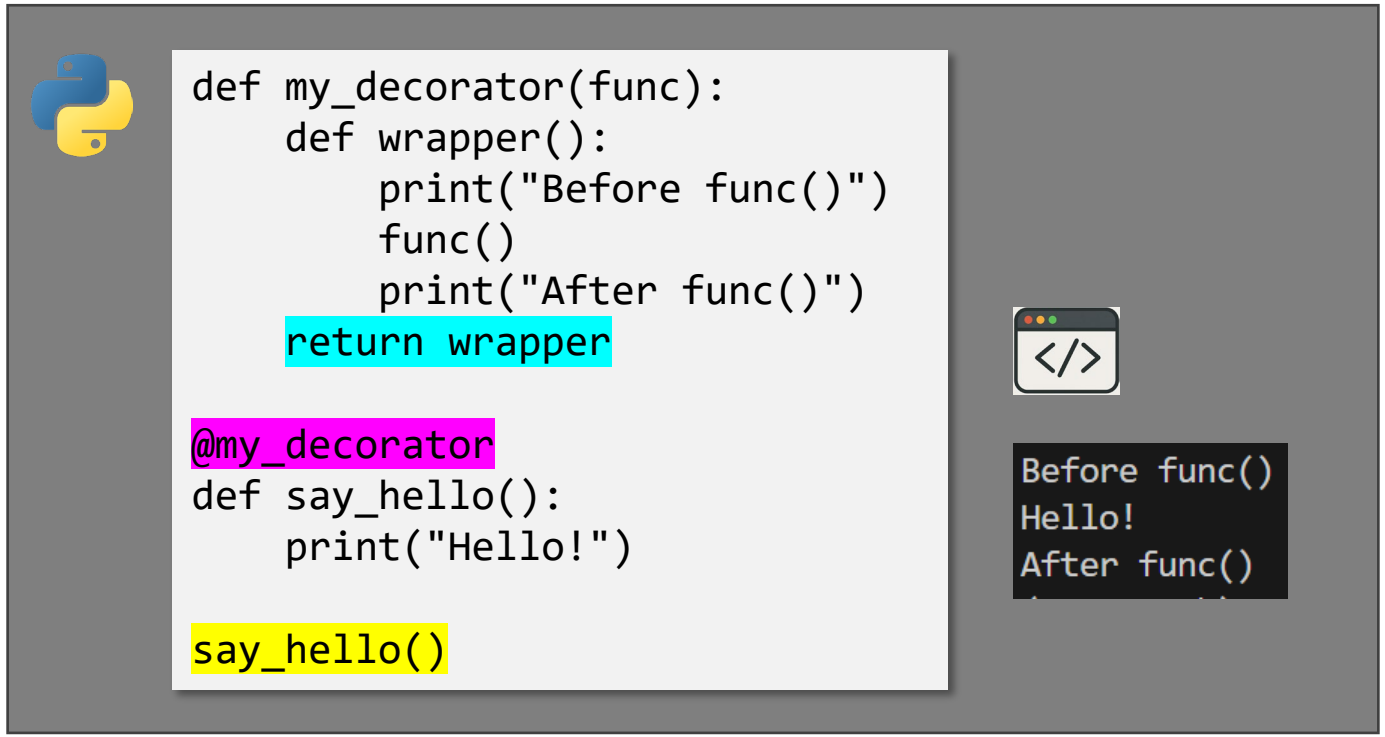
Closure (3)

```python
def my_decorator(func):
    def wrapper():
        print("Before func()")
        func()
        print("After func()")
    return wrapper  # return the closure

def say_hello():
    print("Hello!")

say_hello = my_decorator(say_hello)

# decorated_func is actually 'wrapper'
# which remembers 'func' decorated_func()
say_hello()
```

```
Before func()
Hello!
After func()
```

Decorator

Decorator is a design pattern that uses closures to modify or extend the behavior of functions.



```
def my_decorator(func):
    def wrapper():
        print("Before func()")
        func()
        print("After func()")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

```
Before func()
Hello!
After func()
```

Generator

```
G = (x * 2 for x in range(3))
print(G)
print(list(G))
```

```
<generator object <genexpr> at 0x00000202D0702500>
[0, 2, 4]
```

In `G = (x * 2 for x in range(3))`, the parenthesis () do not create a tuple – they create a generator expression.

Syntax	Example	Result Type	Characteristics
Tuple	(1, 2, 3)	tuple	Stores all elements in memory at once
Generator expression	(x * 2 for x in range(3))	generator	Produces values **one at a time** on demand (lazy evaluation)

Tuple: (value1, value2, ..) → a simple comma-separated list of values is a tuple.
Generator: (for .. in ..) → If there is a for loop inside the parenthesis (), it's a generator expression.

Generator (1)

- Definition: A special kind of **iterator** that doesn't store all values in memory at once.
- Great for memory efficiency, especially when working with very large datasets or potentially infinite sequences.
- **One-time use**: Once a value is consumed, it cannot be retrieved again without recreating the generator.

How to create a generator

(Method 1): Generator Expression

```
G = (x * 2 for x in range(3))  # generator
print(type(G))
print(G)
print(next(G))
print(next(G))
```

```
<class 'generator'>
<generator object <genexpr> at 0x000001D9F98B2500>
0
2
```

(Method 2): Using the yield keyword

```
def my_generator():
    yield 1
    yield 2
    yield 3

g = my_generator()  # generator
print(next(g))  # 1
print(next(g))  # 2
```

```
1
2
```

Generator (2)

Example: Handling large data

```python
def big_numbers():
    for i in range(1_000_000_000):
        yield i

g = big_numbers()    # generator

print(next(g))  # 0
print(next(g))  # 1
# can generate values indefinitely without memory overflow
```

```
0
1
```

List vs Generator

```
# List comprehension
```

```
L = [x**2 for x in range(1_000_000)]  # Uses a lot of memory
```

```
# Generator expression
```

```
G = (x**2 for x in range(1_000_000))  # Uses very little memory
```

A generator is an iterator that produces values **one at a time**, **on demand**, and **without storing them all in memory.**

Thank you!